

Algorithms Analysis

Solving Recurrences



Solving Recurrence

- Solving $T(n) \leq aT(\lceil n/b \rceil) + cn$
 - Solving $T(n) \geq aT(\lceil n/b \rceil) + cn$
 - Solving $T(n) = aT(\lceil n/b \rceil) + cn$
 - Solving $T(n) = aT(\lfloor n/b \rfloor) + cn$
 - Solving $T(n) = aT(\lceil n/b \rceil) + n$
-
- The technique is the same as solving
 - Solving $T(n) = aT(n/b) + n$
 - We will explain this after two weeks.



Solving Recurrence

- If the original one sign is $\leq$,
- solve $T(n)=aT(n/b) + n$, and use Big-Oh to represent your answer.
- If the original one sign is $\geq$,
- solve $T(n)=aT(n/b) + n$, and use Omega to represent your answer.
- If the original one sign is $=$,
- solve $T(n)=aT(n/b) + n$, and use Theta to represent your answer, but it depends on your proof.



Solving $T(n)=2T(n/2) + n$

- Called expansion
- Called recursive method in MIT textbook.
- $T(n)=2T(n/2) + n$
- $=2[2T(n/4)+n/2]+n$
- $=4T(n/4)+2n$
- $=4[2T(n/8)+n/4]+2n$
- $=8T(n/8)+3n$
- ...
- $=2^k T(n/2^k) + kn$, where $2^k = n \rightarrow k = \log n$
- $= nT(1) + n \log n$
- $= \Theta(n \log n)$
- Base case: $T(1)$ is a constant. Needless to say.



Methods for solving recurrences

- Another method:
- Called Mathematical induction
- Called substitution in MIT textbook.
- $T(n) = 2T(n/2) + n$
- Claim: $T(n) < c n \log n$ for some $c > 1$
- For this method, NEVER use big-O or Ω .
- Base case: $n=2$.
- $T(2) = 1 < 2c$ ($T(1) = 1$ is understood, though not given)
- Assume $T(n) < c n \log n$, for $2 \leq n < m$.
- Consider $T(m)$.
- $T(m) = 2T(m/2) + m < 2c(m/2)\log(m/2) + m$
- $= cm \log m - cm + m < cm \log m$.
- Result from induction.



Methods for solving recurrences

- Another method:
- Called Mathematical induction
- Called substitution in MIT textbook.

- $T(n) = 2T(n/2) + n$
- Claim: $T(n) < c n \log n$ for some $c > 1$
- For this method, NEVER use big-O or Ω .

- ~~• Base case: $n=2$.~~
- ~~• $T(2)=1 < 2c$ ($T(1)=1$ is understood, though not given)~~
- Assume $T(n) < c n \log n$, for $2 \leq n < m$.
- Consider $T(m)$.

- $T(m) = 2T(m/2) + m < 2c(m/2)\log(m/2) + m$
- $= cm \log m - cm + m < cm \log m$.

- ~~• Result from induction.~~



Methods for solving recurrences

- Called Mathematical induction
- Called substitution in MIT textbook.
- Simply it further.
- No need to use m.
- $T(n) = 2T(n/2) + n$
- Claim: $T(n) < c n \log n$ for some $c > 1$
- Assume $T(n) < c n \log n$, for $n \geq 2$.
- Consider $T(n)$.
- $T(n) = 2T(n/2) + n < 2c(n/2)\log(n/2) + n$
- $= cn \log n - cn + n < cn \log n$.
- What have we done: $\exists c=2, n_0=2 \bullet \forall n > n_0 \bullet T(n) < cn \log n$.
(no need to mention this if not asked.)
- $T(n) = O(n \log n)$



Methods for solving recurrences

- Solve $T(n)=O(n^2)$ for $T(n)=T(n-1)+n$ by mathematic induction (or substitution).
- In your answer:
- Claim: $T(n) < c n^2$
- Remember even you are asked to prove in terms of big-O notation, you must set up a claim WITHOUT big-O.



Exercises

- Solve the following recurrences:
- $T(n) = T(n/2) + n$, prove $T(n) = O(n)$
- $T(n) = T(2n/3) + n$, prove $T(n) = O(n)$
- $T(n) = 4T(n/2) + n$, prove $T(n) = O(n^2)$
- $T(n) = T(\sqrt{n}) + n$, prove $T(n) = O(n)$
- $T(n) = 5T(\sqrt{n}) + n$, prove $T(n) = O(n)$

The master method

The master method applies to recurrences of the form

$$T(n) = a T(n/b) + f(n) ,$$

where $a \geq 1$, $b > 1$, and f is asymptotically positive.



Three common cases

Compare $f(n)$ with $n^{\log_b a}$:

1. $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$.

- $f(n)$ grows polynomially slower than $n^{\log_b a}$ (by an n^ε factor).

Solution: $T(n) = \Theta(n^{\log_b a})$.

2. $f(n) = \Theta(n^{\log_b a} \lg^k n)$ for some constant $k \geq 0$.

- $f(n)$ and $n^{\log_b a}$ grow at similar rates.

Solution: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.



Three common cases (cont.)

Compare $f(n)$ with $n^{\log_b a}$:

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$.

- $f(n)$ grows polynomially faster than $n^{\log_b a}$ (by an n^ε factor),

and $f(n)$ satisfies the **regularity condition** that $af(n/b) \leq cf(n)$ for some constant $c < 1$.

Solution: $T(n) = \Theta(f(n))$.



Examples

Ex. $T(n) = 4T(n/2) + n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n.$$

CASE 1: $f(n) = O(n^{2-\varepsilon})$ for $\varepsilon = 1$.

$$\therefore T(n) = \Theta(n^2).$$

Ex. $T(n) = 4T(n/2) + n^2$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2.$$

CASE 2: $f(n) = \Theta(n^2 \lg^0 n)$, that is, $k = 0$.

$$\therefore T(n) = \Theta(n^2 \lg n).$$



Examples

Ex. $T(n) = 4T(n/2) + n^3$

$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^3.$

CASE 3: $f(n) = \Omega(n^{2+\varepsilon})$ for $\varepsilon = 1$

and $4(cn/2)^3 \leq cn^3$ (reg. cond.) for $c = 1/2$.

$\therefore T(n) = \Theta(n^3).$

Ex. $T(n) = 4T(n/2) + n^2/\lg n$

$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2/\lg n.$

Master method does not apply. In particular, for every constant $\varepsilon > 0$, we have $n^\varepsilon = \omega(\lg n)$.

